Advanced Probabilistic Machine Learning

Ralf Herbrich, Rainer Schlosser

Exact Inference

Overview

1. Factor Graphs
2. Marginalization by Importance Sampling
3. Marginalization by the Sum-Product Algorithm
4. Practical Considerations in Message Passing
5. Application: Bayesian Regression

Overview

Inference in Probabilistic Models

- **Inference**: In order to learn from data D we follow a three-step procedure
  1. **Modelling**: Formulate a joint model $p(\boldsymbol{\theta}, D)$ of parameters $\boldsymbol{\theta} = \theta_1, \ldots, \theta_n$ and data D
  2. **Conditioning**: Clamp the variables that represent data D (as they are observed)
  3. **Marginalize**: Sum-out all variables that we are not interested in (latent parameters)
- **Example**: Two player game with one winner
  1. **Modelling**: Parameters $\boldsymbol{\theta} = (s_1, s_2, p_1, p_2)$ are skills and performances; data is $y \in \{-1, +1\}$

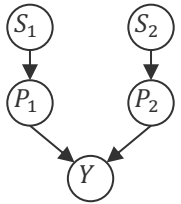
$$p(s_1, s_2, p_1, p_2, y) = \mathcal{N}(s_1; \mu_1, \sigma_1^2) \cdot \mathcal{N}(s_2; \mu_2, \sigma_2^2) \cdot \mathcal{N}(p_1; s_1, \beta^2) \cdot \mathcal{N}(p_2; s_2, \beta^2) \cdot \mathbb{I}(y(p_1 - p_2) > 0)$$

  2. **Conditioning**: Player 1 wins ($y = 1$)

$$p(s_1, s_2, p_1, p_2 | y = 1) \propto \mathcal{N}(s_1; \mu_1, \sigma_1^2) \cdot \mathcal{N}(s_2; \mu_2, \sigma_2^2) \cdot \mathcal{N}(p_1; s_1, \beta^2) \cdot \mathcal{N}(p_2; s_2, \beta^2) \cdot \mathbb{I}(p_1 - p_2 > 0)$$

  3. **Marginalize**: We are only interested in the skills and need to sum-out p_1 and p_2

$$p(s_1, s_2 | y = 1) \propto p(s_1, s_2) \cdot \int_{-\infty}^{+\infty} \int_{-\infty}^{+\infty} \mathcal{N}(p_1; s_1, \beta^2) \cdot \mathcal{N}(p_2; s_2, \beta^2) \cdot \mathbb{I}(p_1 - p_2 > 0) \, \mathrm{d}p_1 \mathrm{d}p_2$$



Factors, Variables and Probabilistic Inference

- **Observation I**: The joint probability model of data and parameters is a *product* of conditional probabilities or clique potentials and has many factors with (few) variables!
- **Observation II**: Conditioning does not reduce factors; it removes variables!
- **Problem**: Naïve summation scales exponentially because we have a sum of products (e.g., product of conditional disitrubtions of all latent variables)!
  - **Example**: Consider an example of n Bernoulli variables $x_1, \dots, x_n$

$$p(x_1) = \sum_{x_2=0}^{1} \sum_{x_3=0}^{1} \cdots \sum_{x_n=0}^{1} p(x_1, x_2, \dots, x_n)$$

2^{n-1} summations

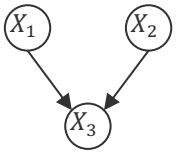
- **Idea**: We exploit the product structure of the probabilisitic model of our data and parameters because not every variable depends on all other variables
  - **Example (ctd)**. Consider $p(x_1, x_2, \dots, x_n) = \prod_i p(x_i)$: then there are only $O(n)$ sums and $n - 1$ sum to one!

Factor Graphs

- **Factor Graph (Frey, 1998)**. *Given a product of m positive functions $f_1, \dots, f_m$, each over a subset of n variables $X_1, X_2, \dots, X_n$, a factor graph if a bipartite graphical model with m factor nodes and n variable nodes where an undirected edge connects f_i and X_j if and only if the function f_i depends on the value of X_j.*
- Factor graphs are more expressive than a Bayesian network and Markov random field!

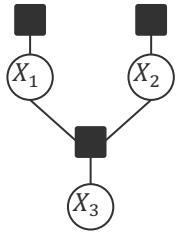
Brendan Frey
(1968 –)

Bayesian network



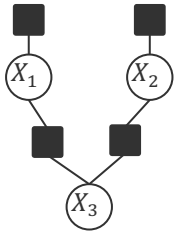
$p(x_1, x_2, x_3) = p(x_1) \cdot p(x_2) \cdot p(x_3|x_1, x_2)$

Corresponding factor graph



$p(x_1, x_2, x_3) = f_1(x_1) \cdot f_2(x_2) \cdot f_3(x_3, x_1, x_2)$

Factor graph with more structure



$p(x_1, x_2, x_3) = f_1(x_1) \cdot f_2(x_2) \cdot f_3(x_1, x_3) \cdot f_4(x_2, x_3)$

Structure in $p(x_3|x_1, x_2)$

Overview

Inference by Importance Sampling

- The key operation is summing-out all but one variable (marginalization).
- **Idea**: If we can get J samples $x_{j,1}, \dots, x_{j,n}, j \in \{1, \dots, J\}$ which are drawn according to

$$p(x_1, \dots, x_n) \propto \prod_{i=1}^{m} f_i(x_1, \dots, x_n)$$

  then we can approximate the marginals $p(x_k)$ arbitrarily well via

$$p(x_k) \approx \frac{1}{J} \sum_{j=1}^{J} \delta(x_k - x_{j,k})$$

- **Challenge**: How do we sample $p(x_1, \dots, x_n)$ if we only have access to (a few) known efficient samplers $q(x_1, \dots, x_n)$ such as (pseudo-random) numbers from the uniform distribution of normal distribution over each X_k?
- **Importance Sampling**: We get J samples $x_{j,1}, \dots, x_{j,n}, j \in \{1, \dots, J\}$ from $q(x_1, \dots, x_n)$ and re-weight them with

$$\frac{p(x_{j,1}, \dots, x_{j,n})}{q(x_{j,1}, \dots, x_{j,n})}$$ ← Importance weight

Weighted-Empirical Distribution

- **Weighted Empirical Distribution**. *Given a sample $\boldsymbol{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$ and n coefficients $\boldsymbol{w} = (w_1, \dots, w_n) \in \mathbb{R}^{+n}$, a random variable $X \in \mathbb{R}$ is said to have the weighted empirical distribution $\mathcal{E}(\cdot; \boldsymbol{x}, \boldsymbol{w})$ if the density is*

$$p(x; \boldsymbol{x}, \boldsymbol{w}) = \frac{1}{Z} \cdot \sum_{i=1}^{n} w_i \cdot \delta(x - x_i) \ , \qquad Z \coloneqq \sum_{j=1}^{n} w_j$$

Normalization constant so that $1/Z \cdot \sum_{i=1}^{n} w_i = 1$

- **Cumulative Distribution Function**. *Let $X \sim \mathcal{E}(\cdot; \boldsymbol{x}, \boldsymbol{w})$ be distributed according to the weighted empirical distribution. Then the cumulative distribution is given by*

$$\int_{-\infty}^{t} \mathcal{E}(x; \boldsymbol{x}, \boldsymbol{w}) \, \mathrm{d}x = \frac{1}{Z} \cdot \sum_{i=1}^{n} w_i \cdot \mathbb{I}(x_i \leq t)$$

Counting all samples that are smaller than t weighing each by $1/Z \cdot w_i$

- **Moments**. *Let $X \sim \mathcal{E}(\cdot; \boldsymbol{x}, \boldsymbol{w})$ be distributed according to the weighted empirical distribution. Then we have for the k-th moment $E[X^k]$*

$$E[X^k] = \frac{1}{Z} \cdot \sum_{i=1}^{n} w_i \cdot x_i^k$$

Weighted sample averages

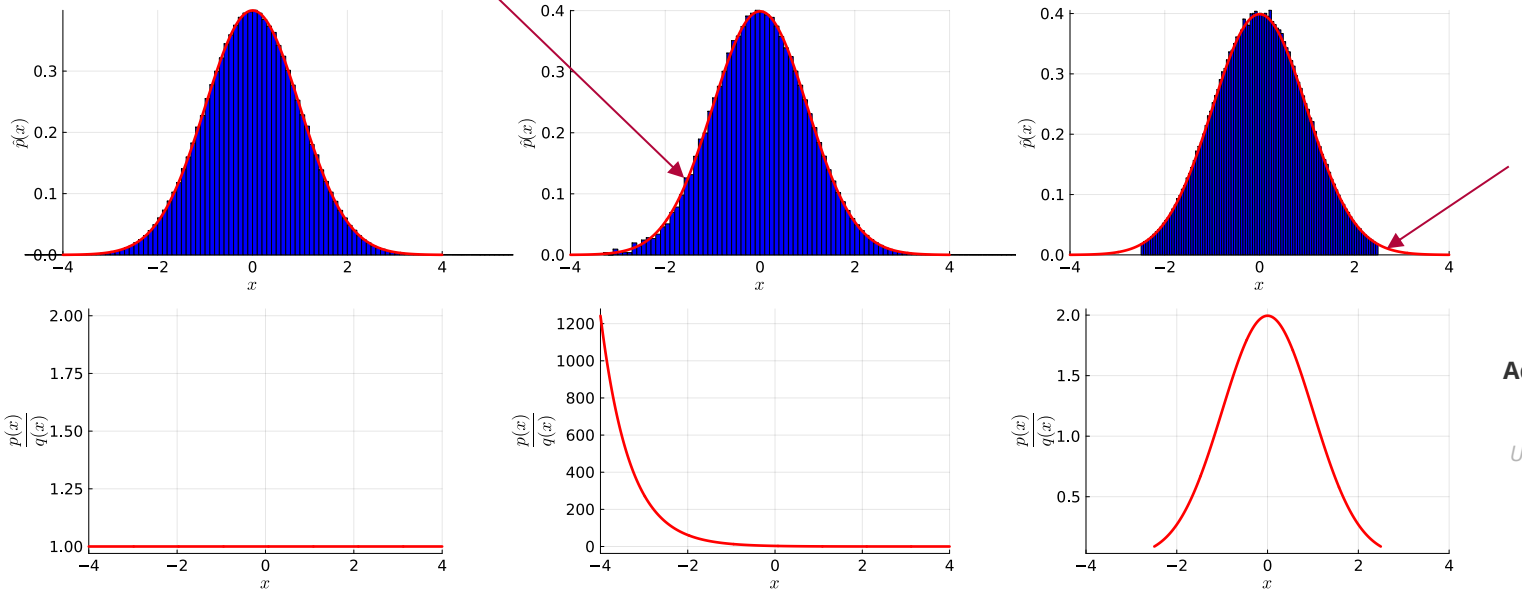
Importance Sampling in Pictures

```
function plot_importance_sampling(p, q; n=100000)
    xs = rand(q, n)
    ws = pdf(p, xs) ./ pdf(q, xs)
    p = histogram(
        xs,
        weights=ws,
        normalize = :pdf,
    )
    xlabel!(L"x")
    ylabel!(L"\hat{p}(x)")
    display(p)
end
```

Too little weight in the proposal

$p(x) = \mathcal{N}(x; 0,1)$

$q(x) = \mathcal{N}(x; 0,1)$ $q(x) = \mathcal{N}(x; 1.5,1)$ $q(x) = \mathcal{U}(x; -2.5,2.5)$



Zero weight in the proposal

Advanced Probabilistic Machine Learning

Unit 4 – Exact Inference

Marginalization by Sampling a Factor Graph

Importance Sampling

Given: Proposal distributions $q_1(\cdot), \dots, q_n(\cdot)$ for $X_1, \dots, X_n$

For $j \in \{1, \dots, J\}$

1. Sample $x_{j,1} \sim q_1$, $x_{j,2} \sim q_2$ up to $x_{j,n} \sim q_n$ into $\boldsymbol{x}_j = (x_{j,1}, \dots, x_{j,n})$
2. Compute

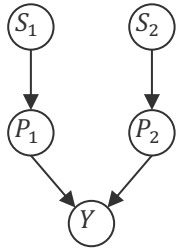
$$w_j = \prod_{i=1}^{m} f_i(x_{j,1}, \dots, x_{j,n}) \Big/ \prod_{k=1}^{n} q_k(x_{j,k})$$

Return: $\{\boldsymbol{x}_j\} \in \mathbb{R}^{J \times n}$ and $\boldsymbol{w} \in \mathbb{R}^{+J}$ as weighted empirical distribution

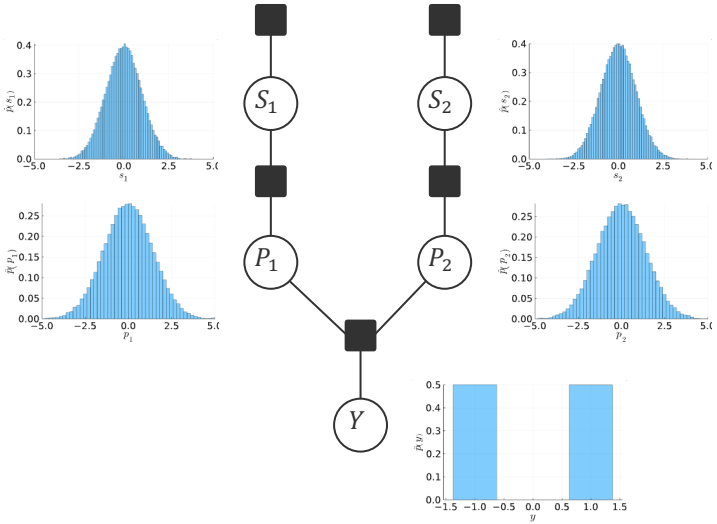
- **Pros**
  1. Sampling of the $\boldsymbol{x}_j$ is parallel rather than sequential (as in a Bayesian network)!
  2. The weights can also be computed in parallel!
- **Cons**
  1. If the proposal $\prod_{k=1}^{n} q_k(\cdot)$ is far from the marginal of $\prod_{i=1}^{m} f_i(\cdot, \dots, \cdot)$ then convergence is slow

Marginalization by Sampling a Factor Graph: Example

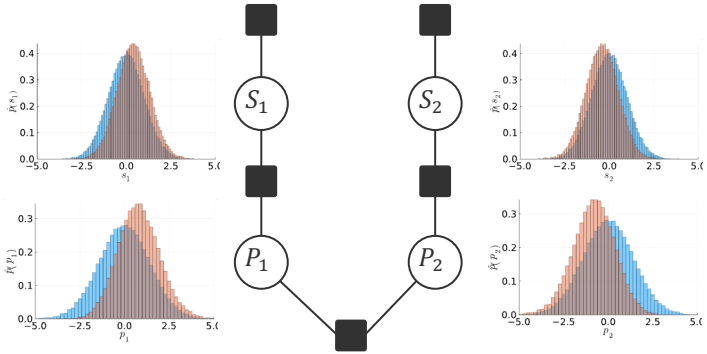
$$\mathcal{N}(s_1; \mu_1, \sigma_1^2) \cdot \mathcal{N}(s_2; \mu_2, \sigma_2^2) \cdot \mathcal{N}(p_1; s_1, \beta^2) \cdot \mathcal{N}(p_2; s_2, \beta^2) \cdot \mathbb{I}(y(p_1 - p_2) > 0)$$



Without match outcome



With match outcome ($y = 1$)



Marginalization by Sampling: Image Denoising Example

- **Conditional Ising Model**: Given a two-color image $y_{i,j} \in \{-1, +1\}$ it is a model for the true color $x_{i,j} \in \{-1, +1\}$ by

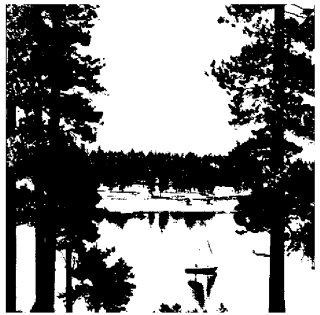
$$p(\boldsymbol{x}|\boldsymbol{y}) \propto \exp\left(\sum_{i=1}^{L}\sum_{j=1}^{L}\left[h \cdot x_{i,j} \cdot y_{i,j} + \sum_{(i',j')\in\mathfrak{N}(i,j)} J \cdot x_{i,j} \cdot x_{i',j'}\right]\right)$$

Coupling strength that a pixel has the same label as its neighbours

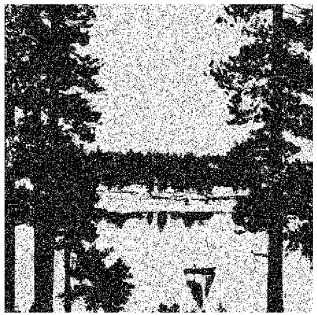
With zero coupling, $h = \frac{1}{2} \cdot \log\left(\frac{p(x_{i,j}=y_{i,j})}{1-p(x_{i,j}=y_{i,j})}\right)$

  - It defines a distribution over images $\boldsymbol{x}$ that only differ from $\boldsymbol{y}$ by a few pixels parameterized via h: Large h are no differences and as $h \to 0$, the fraction converges to 100%

$h = 10, J = 0$ $(\boldsymbol{x} = \boldsymbol{y})$



$h = 1, J = 0$

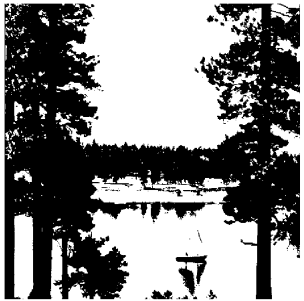


$h = 1, J = 2$

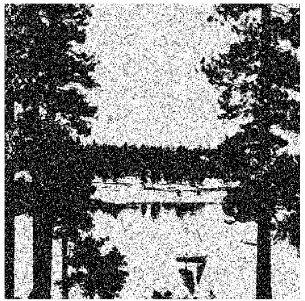
Marginalization by Sampling: Image Denoising Example (ctd)

- **Proposal distribution**: Choose $q(\boldsymbol{x}|\boldsymbol{y}) = p(\boldsymbol{x}|\boldsymbol{y})$ and use Gibbs sampling
  - Then, the weight w_j of each sample is 1 and the importance sampling is just averaging
  - In practice, more data-efficient samplers are used (Markov-Chain Monte Carlo sampler)
- Each marginal $p(x_{i,j})$ is the fraction of Gibbs samples where the pixel is on
- For $J > 0$ the model encodes the smoothness assumptions of areas on the image!

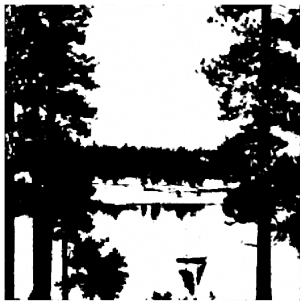
Original $\boldsymbol{y}$



With noise $\boldsymbol{y}'$



$E[\boldsymbol{X}|\boldsymbol{y}']$



Overview

1. Factor Graphs
2. Marginalization by Importance Sampling
3. **Marginalization by the Sum-Product Algorithm**
4. Practical Considerations in Message Passing
5. Application: Bayesian Regression

Marginalization using the Distributive Law

- **Observation 1**: The marginal of a factor graph is a sum (over all values of all hidden variables) of a product (of factor functions).

$$p(x_1) = \sum_{x_2=0}^{1} \sum_{x_3=0}^{1} \cdots \sum_{x_n=0}^{1} f_1(x_1, x_2, \ldots, x_n) \cdot \cdots \cdot f_m(x_1, x_2, \ldots, x_n)$$

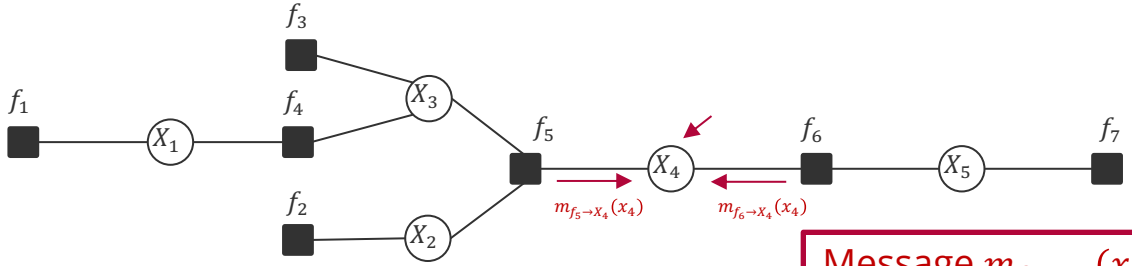
- **Observation 2**: Turning a sum of products *with a common factor* into a product of sums using the *distributive law* saves computation!

$$a \cdot b + a \cdot c = a \cdot (b + c)$$

3 operations — 2 operations

- **Observation 3**: In a typical factor graph, functions only depend on a small number of variables.

Sum-Product Algorithm: Non-normalized Marginals



Message $m_{f_j \to X_i}(x_i)$ is the sum over all variables in the subtree rooted at f_j with $X_i = x_i$

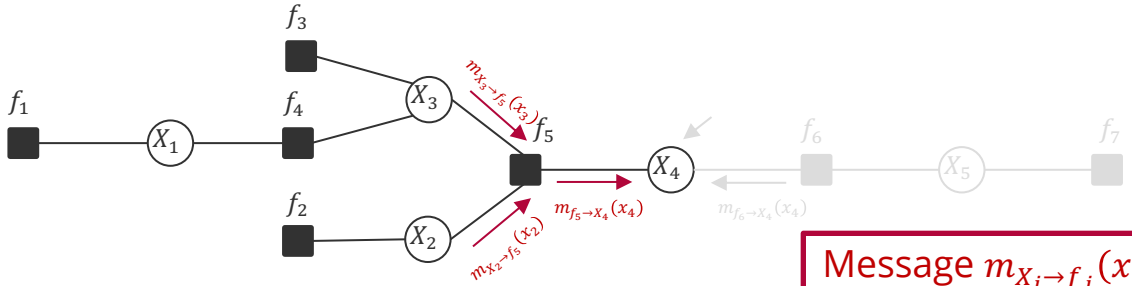
$$p_{X_4}(x_4) = \sum_{\{x_1\}} \sum_{\{x_2\}} \sum_{\{x_3\}} \sum_{\{x_5\}} f_1(x_1) \cdot f_2(x_2) \cdot f_3(x_3) \cdot f_4(x_1, x_3) \cdot f_5(x_2, x_3, x_4) \cdot f_6(x_4, x_5) \cdot f_7(x_5)$$

$$= \left[\sum_{\{x_1\}} \sum_{\{x_2\}} \sum_{\{x_3\}} f_1(x_1) \cdot f_2(x_2) \cdot f_3(x_3) \cdot f_4(x_1, x_3) \cdot f_5(x_2, x_3, x_4) \right] \cdot \left[\sum_{\{x_5\}} f_6(x_4, x_5) \cdot f_7(x_5) \right]$$

$m_{f_5 \to X_4}(x_4)$ $\qquad$ $m_{f_6 \to X_4}(x_4)$

Non-normalized Marginals are the product of all incoming messages from neighbouring factors!

Sum-Product Algorithm: Message from Factor to Variable



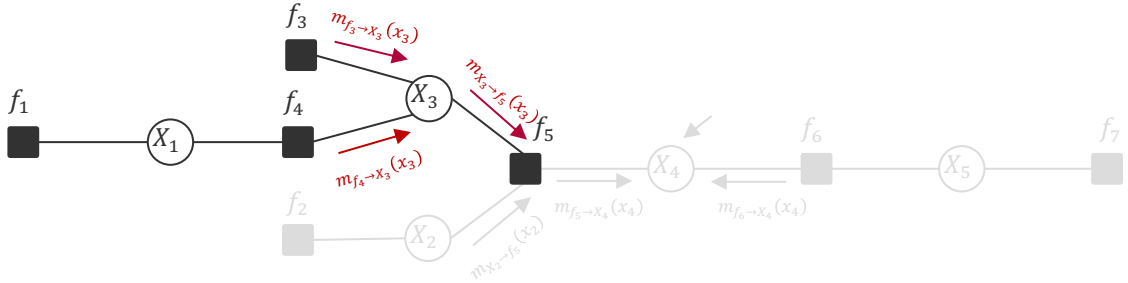
Message $m_{X_i \to f_j}(x_i)$ is the sum over all variables in the subtree rooted at X_i with $X_i = x_i$

$$m_{f_5 \to X_4}(x_4) = \sum_{\{x_1\}} \sum_{\{x_2\}} \sum_{\{x_3\}} f_1(x_1) \cdot f_2(x_2) \cdot f_3(x_3) \cdot f_4(x_1, x_3) \cdot f_5(x_2, x_3, x_4)$$

$$= \sum_{\{x_2\}} \sum_{\{x_3\}} f_5(x_2, x_3, x_4) \cdot \underbrace{[f_2(x_2)]}_{m_{X_2 \to f_5}(x_2)} \cdot \underbrace{\left[\sum_{\{x_1\}} f_1(x_1) \cdot f_3(x_3) \cdot f_4(x_1, x_3)\right]}_{m_{X_3 \to f_5}(x_3)}$$

Messages from a factor to a variable sum out all neighboring variables weighted by their incoming message

Sum-Product Algorithm: Message from Variable to Factor



$$m_{X_3 \to f_5}(x_3) = \sum_{\{x_1\}} f_1(x_1) \cdot f_3(x_3) \cdot f_4(x_1, x_3)$$

$$= \underbrace{[f_3(x_3)]}_{m_{f_3 \to X_3}(x_3)} \cdot \underbrace{\left[\sum_{\{x_1\}} f_1(x_1) \cdot f_4(x_1, x_3)\right]}_{m_{f_4 \to X_3}(x_3)}$$

Messages from a variable to a factor multiply incoming message from neighboring factors

Sum-Product Algorithm

- **Sum-Product Algorithm (Aji-McEliece, 1997)**. Putting it all together, we have

$$p_{X_j}(x_j) = \prod_{i \in \mathrm{ne}(X_j)} m_{f_i \to X_j}(x_j)$$

$$m_{f_i \to X_j}(x_j) = \sum_{\{\boldsymbol{x}_{\mathrm{ne}(f_i)\setminus\{j\}}\}} f(\boldsymbol{x}_{\mathrm{ne}(f_i)}) \prod_{k \in \mathrm{ne}(f_i)\setminus\{j\}} m_{X_k \to f_i}(x_k)$$

$$m_{X_j \to f_k}(x_j) = \prod_{i \in \mathrm{ne}(X_j)\setminus\{k\}} m_{f_i \to X_j}(x_j)$$

- **Basis**: Generalized distributive law (which also holds for max-product)
- **Efficiency**: By storing messages, we
  - Only have to compute local summations in $O(2^T)$ where degree $T = \max_f |\mathrm{ne}(f)|$!
  - All marginals can be computed recursively in $O(E \cdot 2^T)$ vs $O(2^n)$ (where E is the number of edges of the factor graph)!

Robert McEliece
(1942 – 2019)

Overview

1. Factor Graphs
2. Marginalization by Importance Sampling
3. Marginalization by the Sum-Product Algorithm
4. **Practical Considerations in Message Passing**
5. Application: Bayesian Regression

Even more efficiency

- **Redundancies**. By the very definition of messages and non-normalized marginals

$$p_X(x) = \prod_{i \in \mathrm{ne}(X)} m_{f_i \to X}(x) = m_{f_k \to X}(x) \cdot \prod_{i \in \mathrm{ne}(X) \setminus \{k\}} m_{f_i \to X}(x) \longleftarrow m_{X \to f_k}(x)$$

  - **Interpretation**. Application of Bayes' rule at a variable X at factor f

$$p_X(x) = m_{f \to X}(x) \cdot m_{X \to f}(x)$$

non-normalized posterior — Likelihood — non-normalized prior

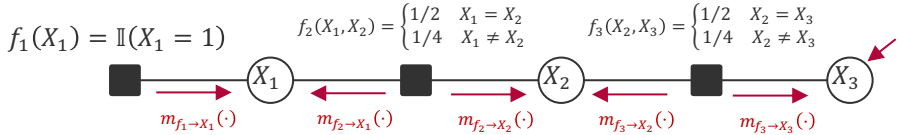
- **Storage Efficiency**. We only store the marginals $p_X(x)$ and $m_{f \to X}(x)$ because

$$m_{X \to f}(x) = \frac{p_X(x)}{m_{f \to X}(x)}$$

- **Exponential Family**. If all the messages from factors to variables are in the exponential family, then the marginals and messages from the variable to factors are simply additions and subtraction of natural parameters (up to normalization)!
  - **Example**: If $p_X(x) = \mathcal{G}(x; \tau_1, \rho_1)$ and $m_{f \to X}(x) = \mathcal{G}(x; \tau_2, \rho_2)$ then $m_{X \to f}(x) \propto \mathcal{G}(x; \tau_1 - \tau_2, \rho_1 - \rho_2)$

A Practical Implementation

1. Initialize all messages $m_{f\to X}(x)$ and marginals $p_X(x)$ with a constant function (i.e., uniform distribution)
2. Pick an arbitrary root (say, X_3)
3. Update all messages $m_{f\to X}(x)$ from the leaves of the tree rooted at X_3 **upwards**
4. Update all messages $m_{f\to X}(x)$ from the root X_3 to the leaves **downwards**



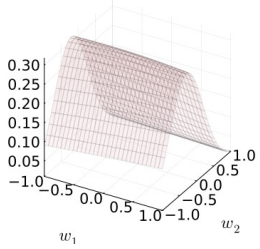
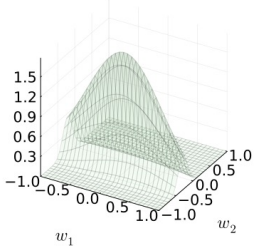
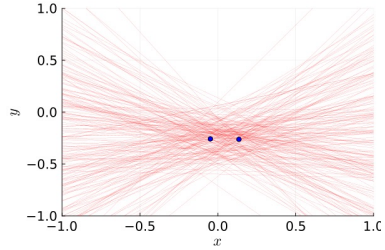
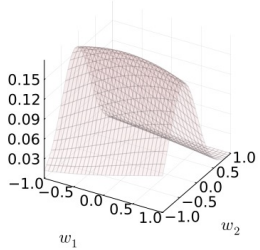
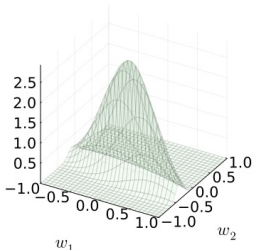
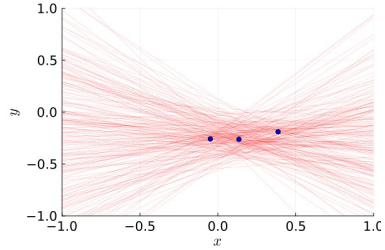
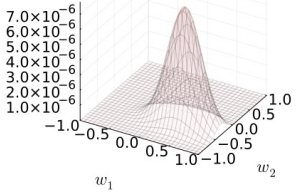
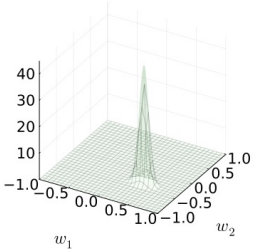
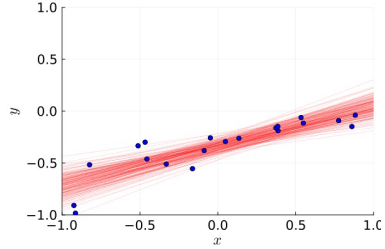
Update	$p_{X_1}(\cdot)$	$p_{X_2}(\cdot)$	$p_{X_3}(\cdot)$	$m_{f_1\to X_1}(\cdot)$	$m_{f_2\to X_1}(\cdot)$	$m_{f_2\to X_2}(\cdot)$	$m_{f_3\to X_2}(\cdot)$	$m_{f_3\to X_3}(\cdot)$
Initial	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$
$m_{f_1\to X_1}(\cdot)$	[1,0,0]	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	[1,0,0]	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$
$m_{f_2\to X_2}(\cdot)$	[1,0,0]	$[\frac{1}{2},\frac{1}{4},\frac{1}{4}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	[1,0,0]	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{2},\frac{1}{4},\frac{1}{4}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$
$m_{f_3\to X_3}(\cdot)$	[1,0,0]	$[\frac{1}{2},\frac{1}{4},\frac{1}{4}]$	$[\frac{3}{8},\frac{5}{16},\frac{5}{16}]$	[1,0,0]	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{2},\frac{1}{4},\frac{1}{4}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{3}{8},\frac{5}{16},\frac{5}{16}]$
$m_{f_3\to X_2}(\cdot)$	[1,0,0]	$[\frac{1}{2},\frac{1}{4},\frac{1}{4}]$	$[\frac{3}{8},\frac{5}{16},\frac{5}{16}]$	[1,0,0]	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{2},\frac{1}{4},\frac{1}{4}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{3}{8},\frac{5}{16},\frac{5}{16}]$
$m_{f_2\to X_1}(\cdot)$	[1,0,0]	$[\frac{1}{2},\frac{1}{4},\frac{1}{4}]$	$[\frac{3}{8},\frac{5}{16},\frac{5}{16}]$	[1,0,0]	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{1}{2},\frac{1}{4},\frac{1}{4}]$	$[\frac{1}{3},\frac{1}{3},\frac{1}{3}]$	$[\frac{3}{8},\frac{5}{16},\frac{5}{16}]$

$$m_{f_2\to X_2}(x_2) = \sum_{x_1=1}^{3} f_2(x_1, x_2) \cdot \frac{p_{X_1}(x_1)}{m_{f_2\to X_1}(x_1)} \longleftarrow m_{X_1\to f_2}(x_1)$$

Overview

1. Factor Graphs
2. Marginalization by Importance Sampling
3. Marginalization by the Sum-Product Algorithm
4. Practical Considerations in Message Passing
5. **Application: Bayesian Regression**

Bayesian Regression in Pictures

	Likelihood	Posterior	Input Space
$n = 2$			
$n = 3$			
$m = 20$			

$$f(x; \boldsymbol{w}) = w_1 x + w_2$$

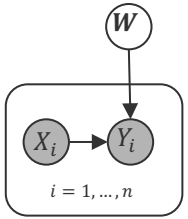
$$p(y|x; \boldsymbol{w}) = \mathcal{N}(y; f(x), 0.2^2)$$

$$p(w_j) = \mathcal{N}(w_j; 0, 0.5)$$

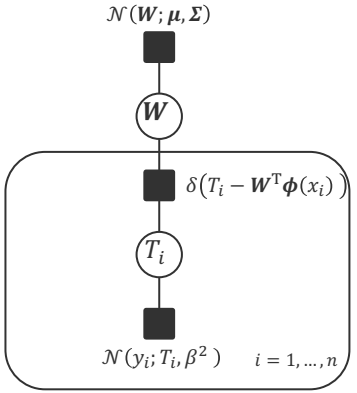
Bayesian Inference of Linear Basis Function Models

- **Given**:
  1. **Training Data**: $D \in (\mathcal{X} \times \mathbb{R})^n$ of n (labelled) examples (x_i, y_i)
  2. **Linear Basis Functions**: Basis function mapping $\boldsymbol{\phi}: \mathcal{X} \to \mathbb{R}^M$ and linear function model
  $$f(x; \boldsymbol{w}) := \boldsymbol{w}^{\mathrm{T}} \boldsymbol{\phi}(x)$$
  weight vector, feature vector
  3. **Likelihood of functions**:
  $$p(D|f) = p(D|\boldsymbol{w}) = \prod_{i=1}^{n} \mathcal{N}\left(y_i; \boldsymbol{w}^{\mathrm{T}} \boldsymbol{\phi}(x_i), \beta^2\right)$$
  4. **Prior belief over functions**:
  $$p(f) = p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$$
- **Bayesian Inference**: Posterior belief over functions
$$p(f|D) = p(\boldsymbol{w}|D) = \frac{\prod_{i=1}^{n} \mathcal{N}\left(y_i; \boldsymbol{w}^{\mathrm{T}} \boldsymbol{\phi}(x_i), \beta^2\right) \cdot \mathcal{N}(\boldsymbol{w}; \boldsymbol{\mu}, \boldsymbol{\Sigma})}{\int_{\mathbb{R}^M} \prod_{i=1}^{n} \mathcal{N}(y_i; \tilde{\boldsymbol{w}}^{\mathrm{T}} \boldsymbol{\phi}(x_i), \beta^2) \cdot \mathcal{N}(\tilde{\boldsymbol{w}}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \, \mathrm{d}\tilde{\boldsymbol{w}}}$$

Bayesian Network



Factor Graph



Multivariate Normal Distribution

- **Multivariate Normal Distribution**. *A continuous random variable* $\boldsymbol{X} \in \mathbb{R}^M$ *is said to have a multivariate normal distribution if the density is given by*

$$p(\boldsymbol{x}) = \frac{1}{\sqrt{(2\pi)^M |\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathrm{T}} \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}-\boldsymbol{\mu})\right)$$

where $\boldsymbol{\Sigma}$ *must be a positive definite* $M \times M$ *matrix and* $\boldsymbol{\mu} \in \mathbb{R}^M$.

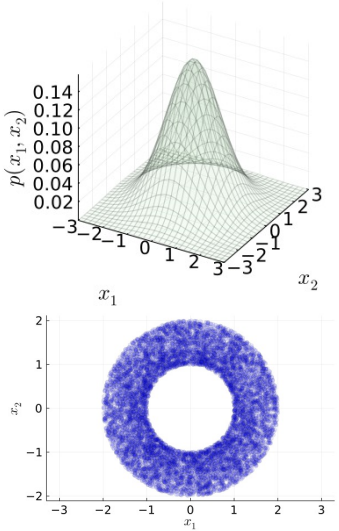
- **Properties**:

$$E[\boldsymbol{X}] = \boldsymbol{\mu}$$
$$\mathrm{cov}[\boldsymbol{X}] = \boldsymbol{\Sigma}$$

- **Covariance**. *For any two random variables* X_1 *and* X_2 *the covariance expresses the extent to which* X_1 *and* X_2 *vary together* **linearly** *and is given by*

$$\mathrm{cov}[X_1, X_2] = E[(X_1 - E[X_1]) \cdot (X_2 - E[X_2])] = E[X_1 X_2] - E[X_1] \cdot E[X_2]$$

  - Generalization of the variance to two random variables: $\mathrm{var}[X] = \mathrm{cov}[X, X]$
  - **Theorem**. *If two random variables* X_1 *and* X_2 *are independent, then* $\mathrm{cov}[X_1, X_2] = 0$. *The converse is not true!*



Advanced Probabilistic Machine Learning

Unit 4 – Exact Inference

Multivariate Normal Distribution: Representations

- **Two Parameterizations (for different purposes)**:
  - **Scale-Location Parameters**

$$\mathcal{N}(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = (2\pi)^{-\frac{M}{2}}|\boldsymbol{\Sigma}|^{-\frac{1}{2}} \cdot \exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right)$$

  - **Natural Parameters**

$$\mathcal{G}(\boldsymbol{x};\boldsymbol{\tau},\boldsymbol{P}) = (2\pi)^{-\frac{M}{2}}|\boldsymbol{P}|^{\frac{1}{2}} \cdot \exp\left(-\frac{1}{2}\boldsymbol{\tau}^{\mathrm{T}}\boldsymbol{P}^{-1}\boldsymbol{\tau}\right) \cdot \exp\left(\boldsymbol{\tau}^{\mathrm{T}}\boldsymbol{x} - \frac{1}{2}\boldsymbol{x}^{\mathrm{T}}\boldsymbol{P}\boldsymbol{x}\right)$$

- **Conversions**

$$\mathcal{N}(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = \mathcal{G}(\boldsymbol{x};\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu},\boldsymbol{\Sigma}^{-1})$$

Matrix inverse

$$\mathcal{G}(\boldsymbol{x};\boldsymbol{\tau},\boldsymbol{P}) = \mathcal{N}(\boldsymbol{x};\boldsymbol{P}^{-1}\boldsymbol{\tau},\boldsymbol{P}^{-1})$$

Multivariate Normal Distributions: Products & Divisions

- **Theorem (Multiplication)**. *Given two multi-dimensional Gaussian distributions* $\mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_1, \boldsymbol{P}_1)$ *and* $\mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_2, \boldsymbol{P}_2)$ *we have*

$$\mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_1, \boldsymbol{P}_1) \cdot \mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_2, \boldsymbol{P}_2) = \mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_1 + \boldsymbol{\tau}_2, \boldsymbol{P}_1 + \boldsymbol{P}_2) \cdot \mathcal{N}(\boldsymbol{\mu}_1; \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2)$$

Gaussian density

Additive updates!

- **Theorem (Division)**. *Given two multi-dimensional Gaussian distributions* $\mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_1, \boldsymbol{P}_1)$ *and* $\mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_2, \boldsymbol{P}_2)$ *where* $\boldsymbol{P}_1 - \boldsymbol{P}_2$ *is positive definite we have*

$$\frac{\mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_1, \boldsymbol{P}_1)}{\mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_2, \boldsymbol{P}_2)} = \frac{\mathcal{G}(\boldsymbol{x}; \boldsymbol{\tau}_1 - \boldsymbol{\tau}_2, \boldsymbol{P}_1 - \boldsymbol{P}_2)}{\mathcal{N}(\boldsymbol{\mu}_1; \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2 - \boldsymbol{\Sigma}_1)} \cdot \frac{|\boldsymbol{\Sigma}_2|}{|\boldsymbol{\Sigma}_2 - \boldsymbol{\Sigma}_1|}$$

Correction factor

Gaussian density

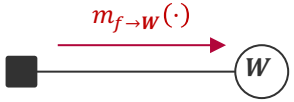
Subtractive updates!

- Natural extension of the one-dimensional multiplication and division rules!

Multivariate Message Update Equations

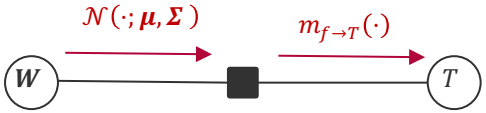
HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam

Gaussian Factor $\mathcal{N}(\boldsymbol{W};\boldsymbol{\mu},\boldsymbol{\Sigma})$



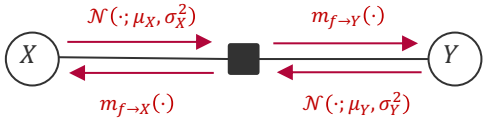
$$m_{f\to \boldsymbol{W}}(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w};\boldsymbol{\mu},\boldsymbol{\Sigma})$$

Gaussian Projection Factor $\delta(T - \boldsymbol{W}^{\mathrm{T}}\boldsymbol{x})$

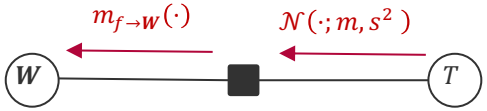


$$m_{f\to T}(t) = \int \delta\left(t - \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}\right)\cdot \mathcal{N}(\boldsymbol{w};\boldsymbol{\mu},\boldsymbol{\Sigma})\ d\boldsymbol{w} = \mathcal{N}\left(t;\boldsymbol{\mu}^{\mathrm{T}}\boldsymbol{x},\boldsymbol{x}^{\mathrm{T}}\boldsymbol{\Sigma}\boldsymbol{x}\right)$$

Gaussian Mean Factor $\mathcal{N}(Y;X,\beta^2)$

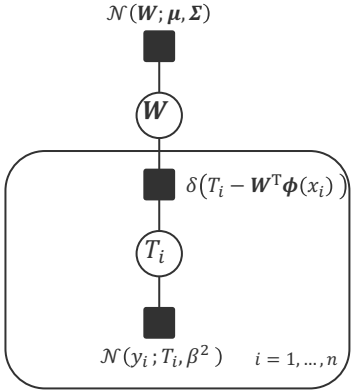


$$m_{f\to Y}(y) = \mathcal{N}(y;\mu_X,\sigma_X^2+\beta^2)$$
$$m_{f\to X}(x) = \mathcal{N}(x;\mu_Y,\sigma_Y^2+\beta^2)$$



$$m_{f\to \boldsymbol{W}}(\boldsymbol{w}) = \int \delta\left(t - \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}\right)\cdot \mathcal{N}(t;m,s^2)\ dt \propto \mathcal{G}\left(\boldsymbol{w};\frac{m}{s^2}\boldsymbol{x},\frac{1}{s^2}\boldsymbol{x}\boldsymbol{x}^{\mathrm{T}}\right)$$

Factor Graph



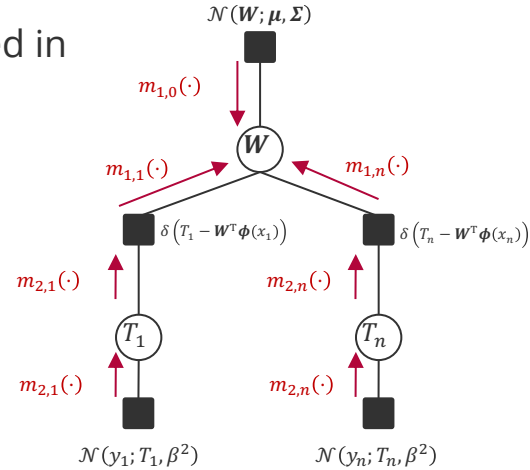
Advanced Probabilistic Machine Learning

Unit 4 – Exact Inference

Bayesian Linear Regression by Message Passing

- **Message**: Simple factor tree where each training example is summarized in an M-dimensional message
  - Prior Message $m_{1,0}(\boldsymbol{w}) = \mathcal{G}(\boldsymbol{w}; \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}, \boldsymbol{\Sigma}^{-1}) = p(\boldsymbol{w})$
  - Target Message $m_{2,i}(t_i) = \mathcal{N}(y_i; t_i, \beta^2) = p(y_i|t_i)$
  - Data Message $m_{1,i}(\boldsymbol{w}) = \mathcal{G}\left(\boldsymbol{w}; \beta^{-2} y_i \boldsymbol{\phi}(x_i), \beta^{-2} \boldsymbol{\phi}(x_i)\boldsymbol{\phi}^{\mathrm{T}}(x_i)\right) = p(y_i|\boldsymbol{w})$
- **Posterior**: Multiplying prior and data messages we have

$$p(\boldsymbol{w}|D) = \mathcal{G}\left(\boldsymbol{w}; \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \beta^{-2}\sum_{i=1}^{n} y_i \boldsymbol{\phi}(x_i), \boldsymbol{\Sigma}^{-1} + \beta^{-2}\sum_{i=1}^{n} \boldsymbol{\phi}(x_i)\boldsymbol{\phi}^{\mathrm{T}}(x_i)\right)$$



- **Feature Matrix**: All feature vectors are stacked on top of each other in a *feature matrix*

feature vector

$$\boldsymbol{\Phi} = \begin{bmatrix} \phi_1(x_1) & \cdots & \phi_M(x_1) \\ \vdots & \ddots & \vdots \\ \phi_1(x_n) & \cdots & \phi_M(x_n) \end{bmatrix} = \begin{bmatrix} \boldsymbol{\phi}^{\mathrm{T}}(x_1) \\ \vdots \\ \boldsymbol{\phi}^{\mathrm{T}}(x_n) \end{bmatrix}$$

$$\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{y} = [\boldsymbol{\phi}(x_1) \quad \cdots \quad \boldsymbol{\phi}(x_n)] \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} = \sum_{i=1}^{n} y_i \boldsymbol{\phi}(x_i)$$

$$\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi} = [\boldsymbol{\phi}(x_1) \quad \cdots \quad \boldsymbol{\phi}(x_n)] \begin{bmatrix} \boldsymbol{\phi}^{\mathrm{T}}(x_1) \\ \vdots \\ \boldsymbol{\phi}^{\mathrm{T}}(x_n) \end{bmatrix} = \sum_{i=1}^{n} \boldsymbol{\phi}(x_i)\boldsymbol{\phi}^{\mathrm{T}}(x_i)$$

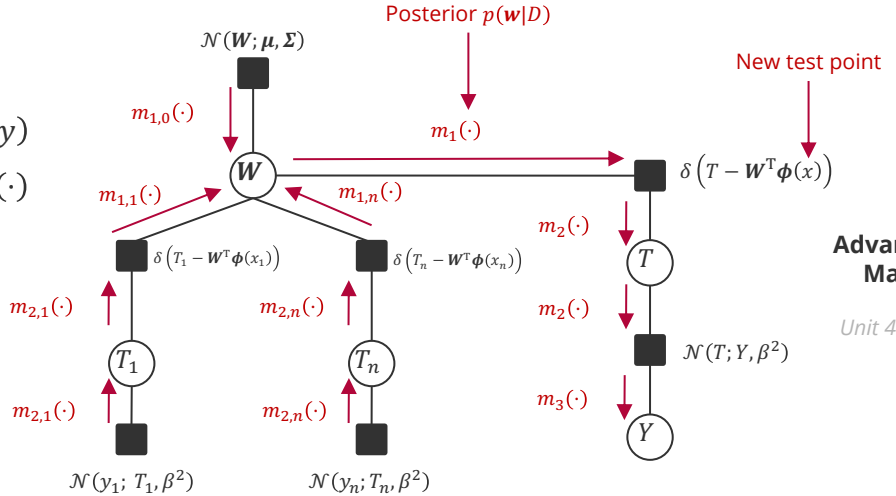
Bayesian Linear Regression: Training & Prediction

- **Posterior**: In terms of the feature matrix, it can be written as

$$p(\boldsymbol{w}|D) = \mathcal{G}\left(\boldsymbol{w}; \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \beta^{-2}\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{y}, \boldsymbol{\Sigma}^{-1} + \beta^{-2}\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}\right)$$
$$= \mathcal{N}(\boldsymbol{w}; \boldsymbol{m}, \boldsymbol{S})$$

$\boldsymbol{S}\left(\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \beta^{-2}\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{y}\right)$ $\quad$ $\left(\boldsymbol{\Sigma}^{-1} + \beta^{-2}\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}\right)^{-1}$

- **Data model for prediction**:
  - Prediction at new test point x is $m_3(y)$
  - Posterior $p(\boldsymbol{w}|D)$ is the message $m_1(\cdot)$ to the Gaussian projection factor at the test point x
  - To avoid recomputing this message for every test point x we simply store the message $m_1(\boldsymbol{w}) = p(\boldsymbol{w}|D)$ as the "model"



Predictions

- **Predicition Tree**: Simple factor chain given posterior $p(\boldsymbol{w}|D) = \mathcal{N}(\boldsymbol{w}; \boldsymbol{m}, \boldsymbol{S})$
  - Posterior Message $m_1(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}; \boldsymbol{m}, \boldsymbol{S}) = p(\boldsymbol{w}|D)$
  - Projection Message $m_2(t) = \mathcal{N}\left(t; \boldsymbol{m}^{\mathrm{T}}\boldsymbol{\phi}(x), \boldsymbol{\phi}^{\mathrm{T}}(x) \cdot \boldsymbol{S} \cdot \boldsymbol{\phi}(x)\right) = p(t|x, D)$
  - Prediction Message $m_3(y) = \mathcal{N}\left(y; \boldsymbol{m}^{\mathrm{T}}\boldsymbol{\phi}(x), \beta^2 + \boldsymbol{\phi}^{\mathrm{T}}(x) \cdot \boldsymbol{S} \cdot \boldsymbol{\phi}(x)\right) = p(y|x, D)$
- **Bayesian Linear Regression in Matrix Notation**

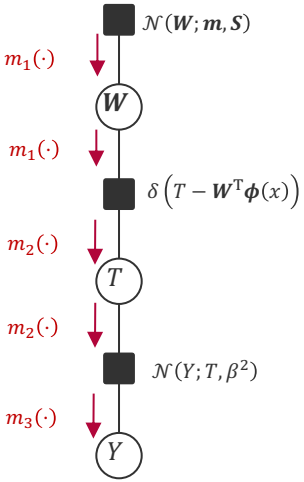
Training

$$\boldsymbol{S} = \left(\boldsymbol{\Sigma}^{-1} + \beta^{-2}\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi}\right)^{-1}$$
$$\boldsymbol{m} = \boldsymbol{S} \cdot \left(\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \beta^{-2}\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{y}\right)$$

Prediction

$$p(y|x, D) = \mathcal{N}\left(y; \boldsymbol{m}^{\mathrm{T}}\boldsymbol{\phi}(x), \beta^2 + \boldsymbol{\phi}^{\mathrm{T}}(x) \cdot \boldsymbol{S} \cdot \boldsymbol{\phi}(x)\right)$$

data uncertainty — model uncertainty



Bayesian Linear Regression: Example

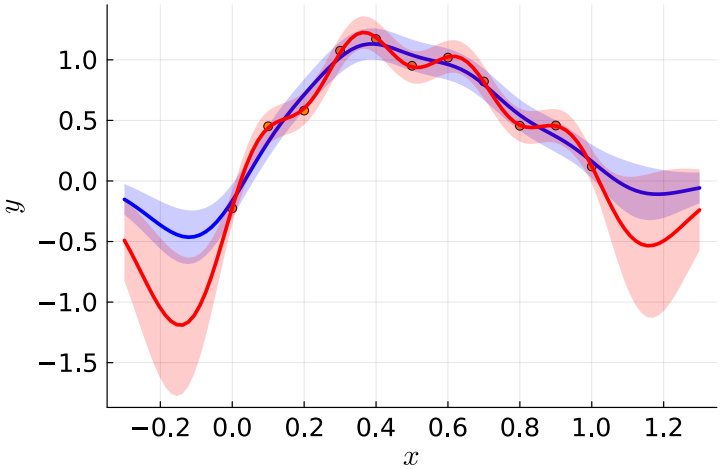
$$p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w}; \boldsymbol{0}, \lambda^2 \boldsymbol{I})$$

$\lambda = 10$

$\lambda = 1$

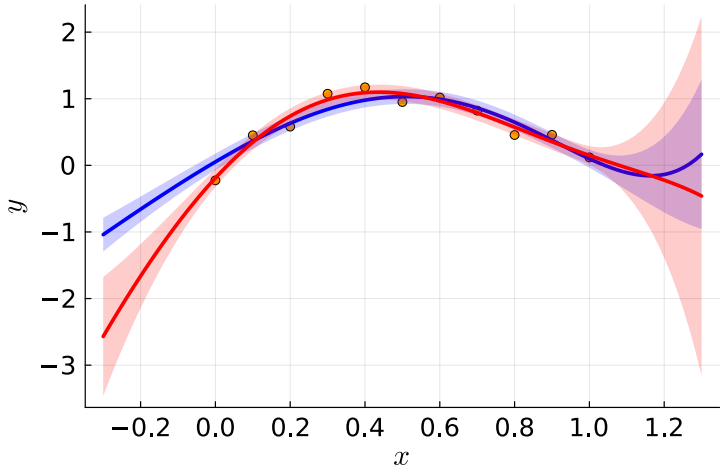
Gaussian Basis

$\phi_j(x) = \mathcal{N}(x; j, 0.15^2)$



Polynomial Basis

$\phi_j(x) = x^j$



Summary

1. **Factor Graphs**
   - Generalization of graphical models specifically designed for fast and easy inference
   - Date back to coding algorithms
2. **Marginalization by Importance Sampling**
   - An easy-to-implement sampling algorithm for the marginal distributions
   - Efficiency depends on the “match” of the proposal distributions with marginal distributions
3. **Marginalization by the Sum-Product Algorithm**
   - Application of generalized distributive law which trades memory (“messages”) for computation (“sums”)
   - Reduces the computational complexity to exponential in the largest out-degree of a factor!
   - There is no faster, exact algorithm for marginalization!
4. **Bayesian Linear Regression**
   - Message passing on the Bayesian Regression factor graph involves no loops and is exact
   - For linear basis function models with Normal noise, the posterior can be computed in closed form as a multivariate Gaussian whose variance accounts for model uncertainty

See you next week!